

Sieci Komputerowe



⌘ Wykład

Wybrane usługi sieciowe w
warstwach wyższych ISO OSI -
- uzupełnienie wcześniejszych
wykładów

HTTP



- ⌘ *HTTP - Hypertext Transfer Protocol* - protokół oparty o tryb zapytanie – odpowiedź:
- ⌘ Główne zastosowanie: wsparcie usług WWW
- ⌘ *URL - Uniform Resource Locator* - podstawowy identyfikator zasobu HTTP, wskazujący obiekty jakie mają być udostępnione w odpowiedzi lub czynności jakie mają być wykonane
- ⌘ Standardowy port TCP/IP dla usług na bazie HTTP: 80, dla wersji szyfrowanej HTTPS: 443
- ⌘ Funkcjonalność HTTP Keepalive – rozszerzenie HTTP umożliwiające utrzymanie trwałego połączenia TCP dla wielu zapytań HTTP (aktywne domyślnie od HTTP 1.1, w HTTP 1.0 wymuszane poprzez podanie polecenia *Connection: Keep-Alive* w nagłówkach HTTP).

HTTP a WWW



- ⌘ Technika odtworzenia dokumentu WWW u klienta:
 - ⏏ Klient kontaktuje się z serwerem wysyłając zapytanie z danym URL
 - ⏏ Serwer wysyła dokument główny (np. w HTML)
 - ⏏ Klient parsuje dokument zawierający linki do składowych dokumentu (grafika, dźwięk, obiekty osadzone, obiekty w ramkach itp.). Następnie wysyła żądanie HTTP dotyczące każdego z tych elementów. Buduje dokument docelowy (np. reprezentowaną graficznie stron WWW) otrzymując jego składniki w toku indywidualnych zapytań HTTP - każdorazowo wymagających połączenia (gdy brak keepalive), zapytania i odpowiedzi.
 - ⏏ Zapytanie i odpowiedź specyfikowane są tzw. tagami HTTP (komendami tekstowymi przekazywanymi poprzez strumień TCP). W przypadku odpowiedzi z treścią – wysłanie komendy pustej (pusta linia) rozpoczyna w strumieniu faktyczną treść.

HTTP - przykład (I)

- ⌘ Wyślano zapytanie: `http://www.agh.edu.pl/user.php?ID=MT`
- ⌘ Zapytanie wysyłane przez klienta HTTP:
 - ⏏ *GET /user.php?ID=MT HTTP/1.1*
 - ⏏ *Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, application/vnd.ms-excel, application/msword, application/x-shockwave-flash*
 - ⏏ *Accept-Language: pl*
 - ⏏ *Accept-Encoding: gzip*
 - ⏏ *User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)*
 - ⏏ *Host: www.agh.edu.pl*

HTTP - przykład (II)

⌘ Odpowiedź serwera:

- ⊞ HTTP/1.1 200 OK
- ⊞ Date: Thu, 01 Apr 2004 17:50:28 GMT
- ⊞ Server: Apache/1.3.29 (Unix) mod_ssl/2.8.16 OpenSSL/0.9.7a
- ⊞ Last-Modified: Wed, 31 Mar 2004 10:26:47 GMT
- ⊞ ETag: "48d5-c07-406a9ce7"
- ⊞ Accept-Ranges: bytes
- ⊞ Content-Length: 3079
- ⊞ Keep-Alive: timeout=15, max=100
- ⊞ Content-Type: text/html
- ⊞ CRLF CRLF
- ⊞ Dane - właściwy dokument HTTP

HTTP - metody



- ⌘ GET – pobranie zasobu wskazanego przez URL
- ⌘ POST – transmisja rozległych danych tekstowych w kierunku od klienta do serwera HTTP
- ⌘ PUT – transmisja danych w postaci pliku w kierunku od klienta do serwera
- ⌘ HEAD – pobranie informacji o zasobie, sprawdzenie jego dostępności
- ⌘ DELETE – polecenie usunięcia zasobu (po autoryzacji)
- ⌘ OPTIONS – przekazanie informacji technicznych dotyczących HTTP (np. możliwość użycia szyfrowania, kompresji itp.)
- ⌘ TRACE – zapytanie diagnostyczne
- ⌘ CONNECT – tryb umożliwiający połączenia serwer → serwer (tunel HTTP)

HTTP – użycie metody GET, POST

- ⌘ POST - parametry (nazwy zmiennych, wartości) są wysyłane jako część nagłówka HTTP (definiowane w kolejnych liniach nagłówka jako niezależne dane). Stosowana w bardziej złożonych przypadkach, gdzie przekazujemy dużą ilość danych w postaci tekstu.
- ⌘ GET - nazwy zmiennych i ich wartości są doklejane do URL.
Przykład:
`http://x.pl/plik.php?zmienna1=wartosc1&zmienna2=wartosc2`
Ograniczeniem tej metody jest długość adresu, zależna np. od typu przeglądarki. Całość przesyłanej z klienta informacji jest jawnie prezentowana użytkownikowi (w linii zapytania). Możliwa jest edycja zapytania przez użytkownika usługi WWW (parametry), przechowywanie kompletnego zapytania (bookmarks).

HTTP - interfejs CGI



- ⌘ CGI (*Common Gateway Interface*) -standard interfejsu służącego wymianie informacji między serwerami WWW a zewnętrznymi programami klientów,
- ⌘ Na podstawie URL serwer uruchamia zewnętrzny program i przekazuje do niego parametry (zmienne środowiskowe) otrzymane z przeglądarki użytkownika,
- ⌘ Program realizuje zlecone operacje,
- ⌘ Wyjście z programu jest wysyłane bezpośrednio do klienta (pipe),
- ⌘ Konieczność pobierania parametrów z linii zapytania URL przy użyciu niestandardowej biblioteki,
- ⌘ Konieczność konwersji wartości parametrów dla przywrócenia w nich znaków zabronionych w URL.

HTTP proxying

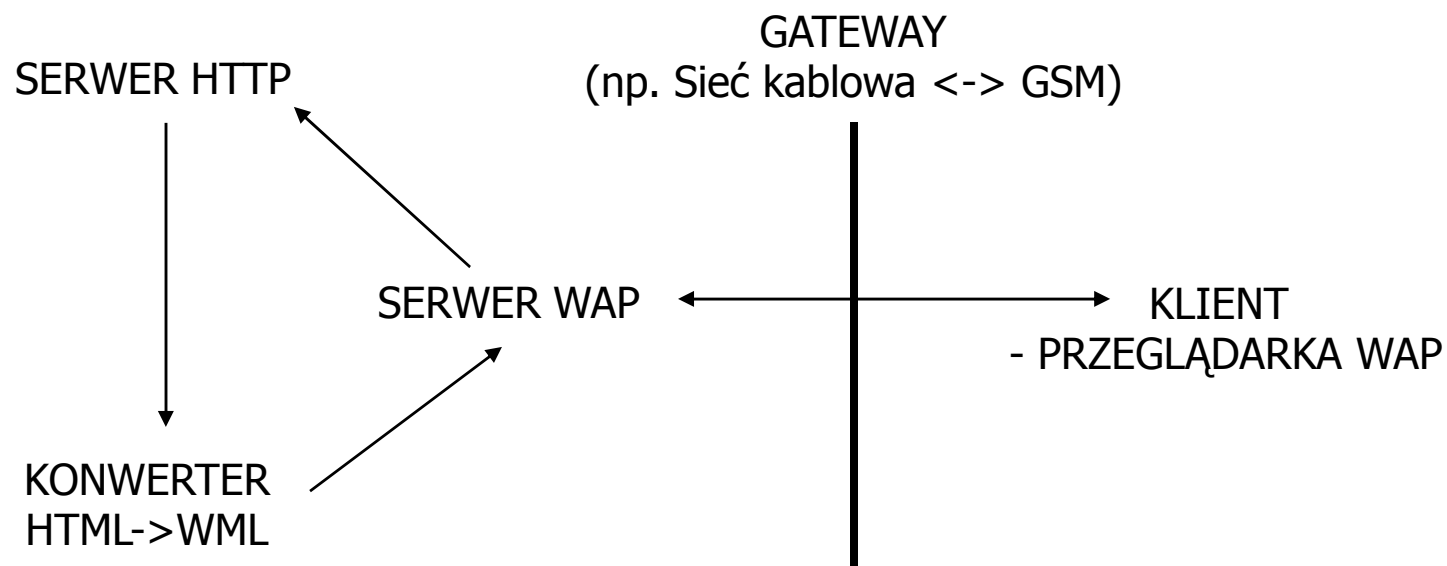
- ⌘ Zastosowanie serwera pośredniczącego w ruchu pomiędzy klientem i serwerem HTTP. Wiele wariantów:
 - ☒ Translation proxy – konwersje URL lub odpowiedzi z adaptacją do cech klienta (np. położenie geograficzne)
 - ☒ Caching proxy – poprawa wydajności obsługi
 - ☒ Anonymous proxy – zatajenie danych o lokalizacji klienta poprzez pośredniczenie w zapytaniach
 - ☒ Security proxy – wymuszenie reguł korzystania z usług (np. eliminowanie *cross-domain resources* czy wykluczenie wytypowanych zapytań, logowanie ruchu itp.)
- ⌘ Implementacje na bazie: SOCKS (niezależny pośrednik), CGI (dodatek do serwera), proxy transparent (przechwycenie ruchu na trasie do serwera HTTP), web proxy (moduł do serwera HTTP), sieć Tor (*The Onion Router*) – globalny wielowęzłowy system tunelowania szyfrowanego wymuszający anonimowość klienta

WAP



- ⌘ *WAP - Wireless Application Protocol* - protokół obsługi aplikacji internetowych uruchamianych na urządzeniach przenośnych
- ⌘ Technologia upadła z rozwojem możliwości urządzeń mobilnych
- ⌘ Protokół technologicznie analogiczny do HTTP - technika klient-server (serwer stron WAP, klient WAP)
- ⌘ Nowy język opisu strony: WML (*Wireless Markup Language*), wzorowany na HTML,
- ⌘ Sprzętowe mikro-przeglądarki w urządzeniach klientów - dokumenty podzielone na buforowane „karty”
- ⌘ Dostosowanie do niewielkich transferów do urządzeń przenośnych - w przypadku telefonów komórkowych 9600 Kbps
- ⌘ Przesył niewielkich pakietów informacji prezentowanych opisem języka - w prostej oprawie graficznej

WAP – schemat działania



FTP (I)



- ⌘ FTP (*File Transfer Protocol*) - protokół transmisji plików
- ⌘ Dwukierunkowy przesył plików zarówno tekstowych jak i binarnych
- ⌘ Funkcjonalność w zakresie transferu informacji o plikach, zarządzania plikami i katalogami
- ⌘ Połączenie na bazie TCP, typowy port połączenia kontrolnego: 21
- ⌘ Dwa połączenia: kontrolne i służące do przekazywania informacji o plikach (oraz samych plików) połączenie danych
- ⌘ Interakcyjny użytkownik wywołuje proces-klienta protokołu FTP w swoim systemie lokalnym, za jego pośrednictwem łączy się z serwerem na zdalnej maszynie

FTP (II)



- ⌘ Tryb tekstowy i binarny transmisji danych
- ⌘ Timeout dla połączenia FTP
- ⌘ Tryb pasywny i aktywny klienta. W trybie aktywnym klienta serwer po otrzymaniu żądania przesyłu danych przesyła je na gniazdo utworzone przez klienta (o numerze portu jest informowany protokołem). W trybie pasywnym - serwer sam zakłada gniazdo, a łączy się na nie klient.
- ⌘ Anonimowe ftp - umożliwienie zalogowania się użytkownika standardowego o nazwie anonymous i korzystanie z udostępnionych publicznych zasobów w ramach obowiązujących praw dostępu dla takich użytkowników

Klient FTP



- ⌘ Klasyczny interfejs tekstowy (CLI) lub klient graficzny
- ⌘ Praca w trybie sesji - otwarcie połączenia, realizacja komend, zamknięcie połączenia)
- ⌘ Komunikacja interakcyjna również na linii klient - serwer. Klient wysyła komunikat i czeka na odpowiedź (zgodnie z protokołem FTP)
- ⌘ Polecenia klienta FTP:
- ⌘ Otwarcie połączenia *ftp nazwa-systemu-zdalnego*, lub po uruchomieniu klienta: *open nazwa-systemu-zdalnego*
- ⌘ *close* lub *disconnect* - zamknięcie połączenia
- ⌘ *bye* lub *quit* - zakończenie programu klienta
- ⌘ *ascii* - ustawienie trybu transferu plików na znakowy
- ⌘ *bin(ary)* - ustawienie trybu transferu plików na binarny
- ⌘ *prompt [on/off]* - (wy)łączenie trybu potwierdzania interaktywnego operacji

Polecenia klienta FTP (II)



- ⌘ Operacje na katalogach:
 - ⌘ *cd. katalog-zdalny* - zmiana katalogu
 - ⌘ *cdup* - przejście w górę drzewa katalogów
 - ⌘ *pwd* - podaje nazwę bieżącego katalogu
 - ⌘ *lcd katalog-lokalny* - tak jak *cd.* dla katalogów lokalnych
 - ⌘ *ls [katalog-lokalny]* - wyświetla zawartość katalogu-lokalnego, można podać argument, domyślnie katalog bieżący
-
- ⌘ Ponieważ w programie klienta można operować zarówno na systemie plików lokalnym jak i zdalnym wprowadzono operator (^!), który postawiony jako przedrostek przed większością komend powoduje, że komenda ta dotyczy systemu lokalnego zamiast zdalnego

Polecenia klienta FTP (III)



- ⌘ *get plik-zdalny [plik-lokalny]* pobiera plik zdalny i zapisuje na lokalnej maszynie w bieżącym katalogu pod nazwą *plik-lokalny* (gdy nie ma parametru to nie zmienia nazwy),
- ⌘ *put plik-lokalny [plik-zdalny]* to samo w drugą stronę,
- ⌘ *mget pliki-zdalne* - działa na grupie plików określonych za pomocą wzorca metaznakowego (znaków specjalnych: *, ?, [a-d])
- ⌘ *mput pliki-lokalne* - działa na grupie plików określonych za pomocą wzorca metaznakowego (znaków specjalnych: *, ?, [a-d])

Protokół FTP - przykład (I)

- ⌘ connecting to 149.156.96.9 ...
- ⌘ Connected to 149.156.96.9 port 21
- ⌘ 220 AGH ftp server galaxy.uci.agh.edu.pl
- ⌘ USER mitu
- ⌘ 331 Password required for mitu.
- ⌘ PASS (hidden)
- ⌘ 230 User mitu logged in.
- ⌘ PWD
- ⌘ 257 "/home/iagrp/mitu" is current directory.
- ⌘ SYST
- ⌘ 215 UNIX Type: L8
- ⌘ Host type (S): UNIX (standard)
- ⌘ PORT 128,0,0,3,8,105
- ⌘ 200 PORT command successful

Protokół FTP - przykład (II)



- ⌘ LIST
- ⌘ 150 Opening ASCII mode data connection for file list
- ⌘ Received 3981 bytes in 0.4 secs, (9.26 KBps), transfer succeeded
- ⌘ 226 Transfer complete.
- ⌘ TYPE I
- ⌘ 200 Type set to I
- ⌘ PORT 128,0,0,3,8,110
- ⌘ 200 PORT command successful
- ⌘ RETR czynniki.zip
- ⌘ 150 Opening BINARY mode data connection for czynniki.zip (26451 bytes)
- ⌘ Received 26451 bytes in 0.8 secs, (33.94 KBps), transfer succeeded
- ⌘ 226 Transfer complete.

DNS



- ⌘ DNS - *Domain Names System* - system nazw domen
- ⌘ Odzwierciedlenie lokalnej struktury sieci w przestrzeni nazw (*Domain Name Space*)
- ⌘ Logiczna przestrzeń nazw, które nie muszą zawierać sieciowych identyfikatorów, adresów, tras ani podobnych informacji
- ⌘ Transakcje serwerów nazw (*Name Servers*) niezależne od systemu komunikacji
- ⌘ Możliwość stosowania tej samej przestrzeni nazw z różnymi rodzinami protokołów
- ⌘ Rozproszone przechowywanie danych z miejscowymi spisami podręcznymi (*Caching*) dla poprawy wydajności

Składniki DNS

- ⌘ Przestrzeń nazw domen i rekordy zasobów (ang. resource records) - określają drzewiastą przestrzeń nazw i dane związane z nazwami. Każdy węzeł i liść przestrzeni nazw wyznacza zbiór informacji a zapytania są próbami wydobywania właściwego typu informacji z określonego zbioru. W zapytaniu jest określona nazwa domeny i żądany typ zasobu.
- ⌘ Serwer nazw (ang. name server) - program przechowujący informację o strukturze drzewa domen. Serwer nazw ma pełne informacje o podzbiorze przestrzeni domen i wskaźniki do innych serwerów, prowadzące do informacji o dowolnej części drzewa domen.
- ⌘ Resolver - program, które wydobywają dane z serwerów nazw w odpowiedzi na zapytania klientów. Resolvery muszą mieć dostęp do przynajmniej jednego serwera nazw i używać jego informacji do bezpośredniej odpowiedzi na zapytanie lub wykorzystać jego odniesienia do innych serwerów. Resolver jest zwykle procedurą systemową bezpośrednio dostępną dla programów użytkowych, zatem nie jest potrzebny protokół komunikacji między nimi.

DNS, definicje



- ⌘ Strefa (ang. zone) - jednostka, którą można automatycznie rozprowadzać do serwerów, która zapewnia nadmiarową obsługę danych w strefie.
- ⌘ DNS nierekurencyjny - serwer odpowiada używając tylko lokalnej informacji. Odpowiedź zawiera błąd, właściwą odpowiedź lub odniesienie (ang. referral) do innego serwera "bliższego" odpowiedzi. Wszystkie serwery nazw muszą działać w trybie nierekurencyjnym.
- ⌘ DNS rekurencyjny - serwer nazw występuje w roli resolvera i zwraca albo błąd albo odpowiedź, ale nigdy odniesienia. Ta usługa jest opcjonalna w serwerze nazw, który może również ograniczać zbiór klientów, którzy mogą używać trybu rekurencyjnego.
- ⌘ Daną strefę powinny obsługiwać co najmniej dwa serwery nazw.

DNS – efekt wynikowy

- ⌘ Z punktu widzenia użytkownika:
 - ☒ Dostęp do systemu domen odbywa się poprzez zwykłą procedurę biblioteczną lub odwołanie systemowe do lokalnego resolvera nazw DNS. Przestrzeń domen składa się z jednego drzewa i można żądać informacji z dowolnej jego części.
- ⌘ Z punktu widzenia resolvera:
 - ☒ System domen składa się z nieznanej liczby serwerów nazw. Każdy serwer nazw ma jedną lub więcej części całego drzewa danych, a resolver rozpoznaje każdą z tych baz danych zasadniczo jako statyczną.
- ⌘ Z punktu widzenia serwera nazw:
 - ☒ System domen jest złożony z rozłącznych zbiorów danych, zwanych strefami. Ma on kopie niektórych stref, ale musi je okresowo odświeżać korzystając z plików głównych innych serwerów. Serwer nazw musi współbieżnie przetwarzać zapytania, które napływają od resolverów.

DNS (V)

⌘ Rodzaje serwerów nazw:

- ☒ Serwer pierwotny (*Primary Name Server*) - przechowuje na swoim lokalnym dysku zestaw plików administracyjnych, zawierających informacje o odwzorowaniu numerów IP i nazw domenowych dla danej strefy
- ☒ Serwer wtórny (*Secondary Name Server*) - posiada dane automatycznie pobierane z pierwotnego serwera nazw (przy starcie). Jeśli ta operacja się nie powiedzie serwer będzie korzystał z wcześniej zapamiętanych kopii plików administracyjnych
- ☒ Serwer pomocniczy (*Caching-only Server*) - po otrzymaniu żądania odwzorowania nazwy domenowej w numer IP, komunikuje się z serwerem autorytatywnym i zapamiętuje to odwzorowanie w swojej pamięci. Dalsze żądania mogą być już obsługiwać bez konieczności komunikacji z serwerem autorytatywnym.

DNS (VI)



⌘ Dodatkowe funkcje serwerów nazw:

- ☑ Serwery autorytatywne - przechowują najbardziej aktualne i wiarygodne informacje dotyczące danej strefy. Serwerami takimi są serwery pierwotne i wtórne strefy
- ☑ Cache zapytań DNS - po wysłaniu informacji do klienta DNS, który przysłał żądanie, informacja o odwzorowaniu będzie przez pewien czas pamiętana przez serwer nazw i w przypadku przyjścia następnego żądania dotyczącego odwzorowania nazwy z tej samej domeny nasz serwer nazw skomunikuje się wprost z serwerem autorytatywnym tej domeny, bez potrzeby łączenia się z innymi serwerami nazw.

DNS (VII)

⌘ Bazy DNS składają się z puli rekordów (wpisów) o różnym znaczeniu. Wśród wpisów DNS można wyróżnić rekordy (uwaga: poniższe oznaczenia literowe są formalnie przyjętymi oznaczeniami typu rekordu DNS):

- ☒ A - lub rekord adresu (*Address Record*) mapuje nazwę domeny DNS na jej 32bitowy adres IPv4
- ☒ AAAA - rekord adresu IPv6 (*IPv6 Address Record*) mapuje nazwę domeny DNS na jej 128 bitowy adres IPv6
- ☒ CNAME - rekord nazwy kanonicznej (*Canonical Name Record*) ustanawia alias nazwy domeny. Wszystkie wpisy DNS oraz poddomeny są poprawne także dla tej nazwy
- ☒ MX - rekord wymiany poczty (*Mail Exchange Record*) mapuje nazwę domeny DNS na nazwę serwera poczty

DNS (VIII)



⌘ Rekordy DNS - ciąg dalszy:

- ☑ PTR - rekord wskaźnika (*Pointer Record*) mapuje adres IPv4 na nazwę kanoniczną hosta - odwrotna translacja adresów DNS (*Reverse DNS Lookup*)
- ☑ NS - rekord serwera nazw (*Name Server Record*) mapuje nazwę domenową na listę serwerów DNS dla tej domeny
- ☑ SOA - rekord adresu startowego uwierzytelnienia (*Start of Authority Record*) ustala serwer DNS dostarczający autorytatywne informacje o domenie internetowej
- ☑ SRV - rekord usługi (*Service Record*) pozwala na zawarcie dodatkowych informacji dotyczących lokalizacji danej usługi, którą udostępnia serwer wskazywany przez adres DNS

DNS (IX)

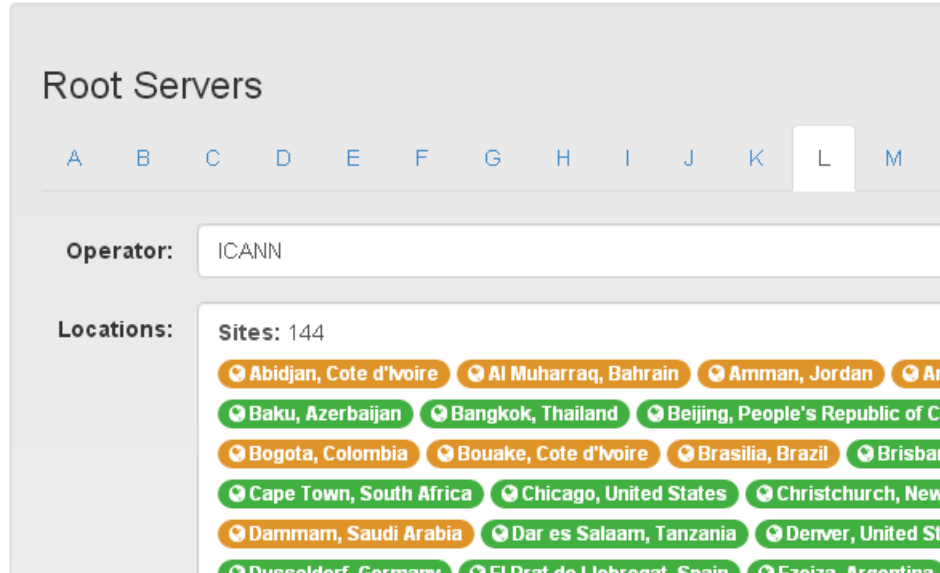
- ⌘ Nazwy domen najwyższego poziomu (TLD - Top Level Domains):
 - ⊞ COM – domena przedsiębiorstw i firm, działalność komercyjna
 - ⊞ EDU – instytucje edukacyjne
 - ⊞ GOV – instytucje rządowe
 - ⊞ MIL – organizacje rządowe
 - ⊞ NET – organizacje prowadzące działalność, związaną z sieciami komputerowymi
 - ⊞ ORG – organizacje o innych charakterach
- ⌘ W nazwach wystąpić mogą dowolne litery, ale porównywanie nazw domen odbywa się bez uwzględnienia wielkości liter
- ⌘ Nazwa musi być unikatowa i nie może przekraczać 255 bajtów
- ⌘ Najwyższy poziom (czyli od prawej) to domena państwa lub kod stanu w przypadku USA

Rejestracja domen i adresów IP

- ⌘ Nadzór nad procesem przyznawania nazw domen z grupy najwyższego poziomu i adresów IP pełnią dwie organizacje:
 - ☒ ICANN (*Internet Corporation for Assigned Names and Numbers*, <http://icann.org>) - obsługa procesu rejestracji domen i IP
 - ☒ IANA (*Internet Assigned Number Authority*, <http://iana.org>) - działa obecnie jako wydział ICANN, sprawuje nadzór techniczny
- ⌘ W Polsce:
 - ☒ Organizacja administrująca domeną .pl - NASK - Naukowo Akademicka Sieć Komputerowa
 - ☒ Aby możliwe było rejestrowanie domeny w pierwszej kolejności konieczne jest posiadanie hosta (rzeczywistego lub wirtualnego) i wpisów w DNS strefy lokalnej

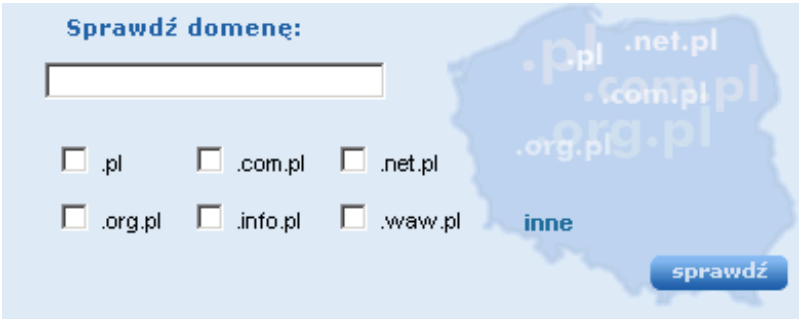
1. *Journal of the American Medical Association*, 1997; 277: 1001-1005.

- 



Procedura rejestrowania domeny w Polsce

- ⌘ Gdy domena jest wolna - złożenie wniosku o delegację domeny wraz z uzasadnieniem. Wniosek dostępny jest pod adresem <http://www.dns.pl/cgi-bin/wniosek.pl>
- ⌘ Delegacja polega na wpisaniu do DNS strefy polskiej adresów DNS lokalnych, gdzie znajduje się informacja o hoscie docelowym
- ⌘ Wyszukiwarka NASK (najbardziej wiarygodna) znajduje się pod adresem: <http://www.dns.pl>
- ⌘ Opłaty za domenę pobierane są przez NASK w rozliczeniu rocznym



© NASK

System poczty elektronicznej



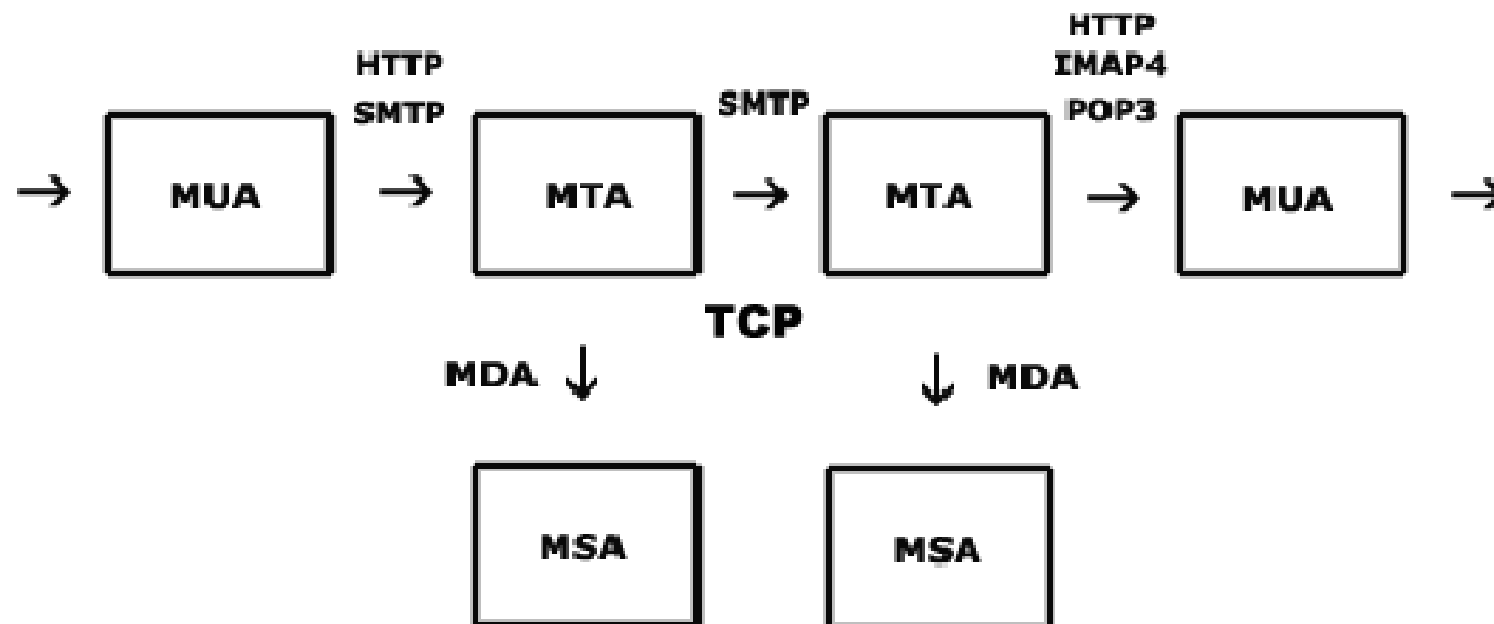
- ⌘ Jedna z najważniejszych usług sieciowych
- ⌘ System pocztowy dostarczany przez TCP/IP zbudowany jest na bazie kilku protokołów:
 - ☒ SMTP - prosty protokół wymiany poczty (*Simple Mail Transfer Protocol*)
 - ☒ POP - protokół pocztowy (*Post Office Protocol*)
 - ☒ IMAP - interaktywny protokół dostępu do poczty (*Interactive Mail Access Protocol*) będący następcą POP
 - ☒ MIME - protokół wielozadaniowych rozszerzeń poczty internetowej (*Multipurpose Internet Mail Extensions*)

System poczty elektronicznej - elementy



- ⌘ Agent przekazywania wiadomości (*Message Transfer Agent, MTA*) - służy do wymiany wiadomości, przetwarza i przesyła w obu kierunkach pomiędzy MSA i MUA. Przykład: Sendmail w Unix
- ⌘ Agent wiadomości użytkownika (*Message User Agent, MUA*) - pozwala użytkownikowi na wysłanie, czytanie i manipulowanie wiadomościami
- ⌘ Agent magazynu wiadomości (*Message Store Agent, MSA*)- nie istnieje standard określający strukturę MSA - implementacje MSA są różne
- ⌘ Agent dostarczania wiadomości (*Message Delivery Agent, MDA*) - odpowiada za dostarczanie wiadomości pocztowych do odpowiednich źródeł.

System poczty elektronicznej (schemat)



SMTP



- ⌘ Protokół umożliwiający dostarczenie poczty elektronicznej w sieci TCP/IP
- ⌘ Wykorzystanie do transmisji niezawodnej, połączeniowej usługi protokołu TCP
- ⌘ Port komunikacyjny: 25
- ⌘ Komunikacja polega na przesyłaniu prostych poleceń od klienta, na które odpowiada serwer pocztowy
- ⌘ Do przesyłania danych zastosowano kod NVT ASCII (7-bitowy kod znakowy, który przesyłany jest w postaci 8-bitowych bajtów z najstarszym bitem ustawionym na 0).
- ⌘ Protokół daje dowolność w dołączaniu porządkowych nagłówek wiadomości - na różnych poziomach przekazu przez MTA

SMTP - format wiadomości



⌘ Wiadomość zawiera trzy części:

- ☒ Koperta – używana jest przez MTA w celu dostarczenia wiadomości. Zawiera przede wszystkim adres nadawcy i odbiorcy
- ☒ Nagłówki – używane są przez agentów użytkownika. Każde z pól nagłówka zawiera w kolejności: nazwę, dwukropek, wartość pola. Format i sposób interpretacji pól nagłówka określony jest w standardowej specyfikacji RFC 822. Nagłówki rozpoczynające się od znaku X są polami zdefiniowanymi przez użytkownika, pozostałe definiuje specyfikacja protokołu (RFC 822).
- ☒ Zawartość – jest tym co nadawca chce przekazać adresatowi - tekst w formacie NVT ASCII

SMTP - lista podstawowych komend

 Polecenie	Składnia	Funkcja :
 Hello	HELLO <host-nadawcy>	Identyfikacja hosta nadającego
 From	MAIL FROM:<adres-nadawcy>	Adres nadawcy
 Recipient	RCPT TO:<adres-odbiorcy>	Adres odbiorcy
 Data	DATA	Początek wiadomości
 Reset	RSET	Anulowanie wiadomości
 Verify	VRFY <łańcuch-znaków>	Weryfikacja nazwy użytkownika
 Noop	NOOP	Zmusza serwer do przesłania OK.
 QUIT	QUIT	Zakończenie sesji SMTP

SMTP - przykład (I)

- ⌘ Ponieważ SMTP jest prostym protokołem umożliwiającym interakcję przy użyciu komunikatów tekstowych, można stworzyć wiadomość za pomocą komend SMTP wysyłanych przez telnet (port 25).
- ⌘ Przykład wiadomości wysłanej przez użytkownika *mitu* z hosta *ernie.icslab.agh.edu.pl* do *mitu2* na *plus.agh.edu.pl*:

⌘ `mitu@plus:~ (22) > telnet ernie 25`

⌘ `Trying 149.156.127.5...`

⌘ `Connected to ernie.`

⌘ `Escape character is '^]'.`

⌘ `220 ernie.icslab.agh.edu.pl ESMTP Sendmail 8.8.7/8.8.5/ts-tmpl.970124+rchk1.22; Sat, 21 Apr 2001 15:08:25 +0200 (MET DST)`

⌘ `hello ernie`

⌘ `250 ernie.icslab.agh.edu.pl Hello mitu@plus.agh.edu.pl [149.156.127.16], pleased to meet you`

SMTP - przykład (II)

⌘ Przykład wysyłania listu - ciąg dalszy (II)

```
⌘ mail from:< mitu@ernie.icslab.agh.edu.pl>
⌘ 250 < mitu@ernie.icslab.agh.edu.pl >... Sender ok
⌘ rcpt to:< mitu2@plus.agh.edu.pl >
⌘ 250 < mitu2@plus.agh.edu.pl >... Recipient ok
⌘ data
⌘ 354 Enter mail, end with "." on a line by itself
⌘ test
⌘ .
⌘ 250 PAA24739 Message accepted for delivery
⌘ quit
⌘ 221 ernie.icslab.agh.edu.pl closing connection
⌘ Connection closed by foreign host.
⌘ mitu@plus:~ (23) >
```

SMTP - rozszerzenie MIME



- ⌘ MIME dodaje kilka nowe nagłówki wiadomości (zgodnie z zaleceniami RFC 822) i wymaga aby agenci użytkownika obsługiwali MIME. MIME nie wymaga wprowadzania żadnych zmian do MTA
- ⌘ Nowe nagłówki:
 - ☒ Content-Transfer-Encoding – inny sposób kodowania zawartości
 - ☒ Content-Type – definiuje typ danych wiadomości:
 - Text
 - Multipart
 - Message
 - Application
 - Image
 - Audio
 - Video

Sendmail – popularny serwer pocztowy



- ⌘ Unifikacja programów obsługi i protokołów pocztowych
- ⌘ Funkcjonalność:
 - ☒ Przesyłanie poczty pomiędzy różnymi programami pocztowymi w systemie unix/linux
 - ☒ Odbieranie i wysyłanie wiadomości przez SMTP
 - ☒ Utrzymywanie obowiązujących w całym systemie aliasów, co pozwala na tworzenie list adresowych
- ⌘ Sendmail należy uruchomić jako demona. Będzie on nasłuchiwał na porcie TCP 25 i przetwarzał przychodzące listy

POP



- ⌘ Protokół pozwala na przesyłanie poczty od serwera pocztowego do lokalnego programu pocztowego użytkownika (tzw. „pobieranie poczty”)
- ⌘ Weryfikacja nazwy użytkownika i jego hasła
- ⌘ Protokół rozwinięto do wersji POP2, POP3 (obecna)
- ⌘ POP2 używa portu TCP/109, a POP3 TCP/110
- ⌘ Podobnie jak i w przypadku SMTP polecenia protokołu POP można zapisywać do odpowiedniego portu TCP nawiązując wcześniej z połączenie za pomocą telnet

POP3 - lista podstawowych komend



⌘ USER nazwa-uzytkownika	Nazwa konta użytkownika
⌘ PASS hasło	Hasło użytkownika
⌘ STAT	Wyświetla liczbę nieodebranych wiadomości i ich objętość w bajtach
⌘ RETR n	Odbiera wiadomość o numerze n
⌘ DELE n	Usuwa wiadomość o numerze n
⌘ LAST	Wyświetla numer wiadomości na której ostatnio wykonano polecenie
⌘ LIST[n]	Wyświetla objętość wszystkich wiadomości lub n
⌘ TOP n l	Wyświetla nagłówki i l pierwszych linii wiadomości n
⌘ NOOP	Nie podejmuje akcji (diagnostyka)
⌘ QUIT	Kończy sesję POP3

POP3 - przykład

- ✘ **telnet** *galaxy.uci.agh.edu.pl 110*
- ✘ *Trying 149.156.96.9.110...*
- ✘ *Connected to galaxy.uci.agh.edu.pl.*
- ✘ *Escape character is '^]'.*
- ✘ *+OK QPOP (version 3.0.2) at galaxy starting.*
- ✘ **user** *mitu*
- ✘ *+OK Password required for mitu.*
- ✘ **pass** *mojehaslo:)*
- ✘ *+OK mitu has 23 visible messages (0 hidden) in 57094 octets.*
- ✘ **stat**
- ✘ *+OK 23 57094*
- ✘ **list**
- ✘ *+OK 23 visible messages (57094 octets)*
- ✘ *1 2482*
- ✘ *2 2430*
- ✘ *.*
- ✘ **retr** *2*
- ✘ *+OK 2430 octets*
 - <... TU ZOSTANĄ WYŚWIETLONE SZCZEGÓŁY WIADOMOŚCI.

IMAP



- ⌘ Jest odpowiednikiem protokołu POP z rozbudowaną funkcjonalnością
- ⌘ Pozwala na synchronizację skrzynki pocztowej
- ⌘ Możliwość manipulowania pojedynczymi wiadomościami zarówno w systemie serwera, jak i w systemie klienta. Zmiany dokonane w jednym z tych systemów zostaną uwzględnione w skrzynkach pocztowych obydwu
- ⌘ IMAP wykorzystuje port TCP 143
- ⌘ Protokół rozwinięto do wersji IMAP2, IMAP3, IMAP4 (obecna)
- ⌘ W systemie UNIX - demon *imapd* - obsługuje serwer pocztowy z użyciem IMAP

Protokołu Internetu dla transmisji multimedialnych



- ⌘ Udostępnianie sekwencji wideo. Tryby przekazu strumienia video:
 - ☒ Przesył na żądanie - serwery mają na swoich dyskach zapisane wcześniej dane wizyjne,
 - ☒ Rozsyłanie automatyczne,
 - ☒ Jednokierunkowe sekwencje "na żywo" do wielu adresatów - mała interakcja,
 - ☒ Przesył dwukierunkowy.

Multimedia w sieci (II)

⌘ Standardy wideokonferencji:

- ☒ H.320 - standard wąskopasmowy (np. dla ISDN), duża kompresja obrazu na podstawie algorytmów stanowiących integralny element standardu. Pasma wynikowe : 64 Kb/s
- ☒ H.310 i H.321 - przystosowują H.320 do topologii, takich jak ATM i szerokopasmowe ISDN. Są kompatybilne wstecz.
- ☒ T.120 jest standardem konferencji dokumentowych, umożliwiającym użytkownikom wspólną pracę nad dokumentami podczas dowolnej wideokonferencji H.32x

Multimedia w sieci (III)

⌘ Voice over IP:

- ☒ Telefonia na bazie transmisji z komutacją pakietów
- ☒ Stosowanie bramy telefonicznej (*phone gateway network*) pełniącej rolę serwera online i zestawiającej połączenia także ze zwykłą siecią telefoniczną
- ☒ Z zastosowaniem bramki możliwa jest rozmowa nie tylko na linii host-host ale także host-telefon, telefon-host, łącznie z automatycznym wywołaniem
- ☒ Zadowalająca jakość jest uzyskiwana przy przepustowości minimum 8 kbps (bez kompresji)
- ☒ Uzgodnień standaryzacyjnych co do techniki kompresji i parametrów prędkości transmisji
- ☒ Kluczowy problem - opóźnienia w dostawie pakietów strumienia audio

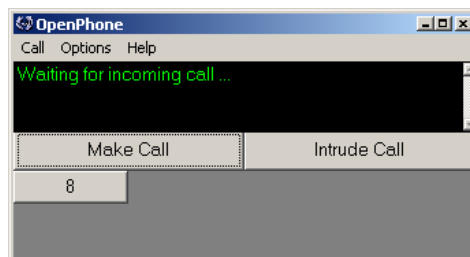
Standard H.323



- ⌘ Stos protokołów H.323, zawierający rozwiązania dotyczące kontroli rozmieszczenia urządzeń, formatowania strumieni komunikacyjnych, bezpieczeństwa itp.
- ⌘ Opisuje techniczne wymagania dla usług łączności audio i wideo w czasie rzeczywistym poprzez sieci transmisji pakietowej
- ⌘ Zalecenia H.323 definiują kompleksowo architekturę sieci, która umożliwia między innymi realizację połączeń telefonicznych w sieciach pakietowych
- ⌘ Standard operuje w warstwie transportowej i wyższych.

H.323 - elementy systemu

- ⌘ Terminal – telefon internetowy lub odpowiednie oprogramowanie komunikacyjne, które realizuje algorytmy kompresji i dekompresji danych oraz inne funkcje umożliwiające porozumiewanie się użytkowników zgodnie ze stosem protokołów (dialing, obsługa interfejsu IP, negocjowanie parametrów transmisji multimedialnych, rejestrowanie w usługach VoIP itp.).
- ⌘ Bramka VoIP (*VoIP Gateway*) - urządzenie pośredniczące w komunikacji systemów końcowych H.323 z innymi systemami (translacja sygnału analogowego na strumień multimedialny, zarządzanie numerami terminali w bramce). Do bramki podłączone są bezpośrednio np. telefony analogowe.



H.323 - elementy systemu



- ⌘ Gatekeeper - serwer sterujący, który zarządza połączeniem, zapewnia translację adresów (numery telefonów na IP lub IPX), autoryzację i zarządzanie szerokością przydzielonego pasma dla danej transmisji.
- ⌘ Mostek konferencyjny MCU (*Multipoint Control Unit*) - służący do sterowania transmisją multicast
- ⌘ Call Manager – oprogramowanie będące rozszerzeniem GateKeeper i dostarczające rozszerzenia funkcjonalne: informacje statusie rozmówcy, książki telefoniczne, pocztę głosową itp.

H.323 - ważniejsze protokoły (I)



⌘ H.245 - Protokół kontroli dla komunikacji multimedialnej:

- ☒ Master-Slave Determination - określenie, który terminal ma status Master, a który Slave.
- ☒ Terminal Capability Set - informacje o zdolności terminala do przetwarzania strumieni multimedialnych.
- ☒ Keypad Type - typ wybierania (tonowy, alfanumeryczny)
- ☒ Open/Close Logical Channel - sterowanie kanałami dla transmisji multimedialnej
- ☒ Request Mode - typ transmisji (audio, wideo, szyfrowana itp.)

H.323 - ważniejsze protokoły (II)



- ⌘ RAS (*Registration, Admission, Status*) - obsługa sygnalizacji między gatekeeperem, a innymi elementami systemu. Transmisja RAS odbywa się w trybie bezpołączeniowym. Komunikaty:
 - ☒ Registration - prośba od terminala albo bramki o rejestrację w gatekeeperze
 - ☒ Admission - prośba od terminala do gatekeepera o dostęp do sieci pakietowej
 - ☒ Bandwidth - prośba od terminala do gatekeepera o zmianę szerokości przydzielonego pasma

H.323 - ważniejsze protokoły (III)



- ⌘ Q.931 Call Signalling - procedury niezbędne do zrealizowania połączenia między stroną inicjującą a stroną docelową:
 - ⏏ Call Proceeding - zawiadomienie terminala użytkownika o przejściu do fazy zestawiania połączenia
 - ⏏ Connect - odpowiedź na wezwanie, czyli "podniesienie słuchawki"
 - ⏏ Release Complete - zakończenie połączenia, czyli „odłożenie słuchawki"
 - ⏏ Status Inquiry /Status - pytanie o status rozmowy/odpowiedź

H.323 - popularne kodeki audio/video



⌘ Podstawowe parametry konfiguracji:

- ☒ Czas segmentacji pakietów (ustalany z reguły w milisekundach) (np. G.711)
- ☒ Eliminacja ciszy (*Silence Suppression*) - wycinanie fragmentów strumienia, gdzie nic nie jest przekazywane (np. G.723.1)

⌘ Kodeki:

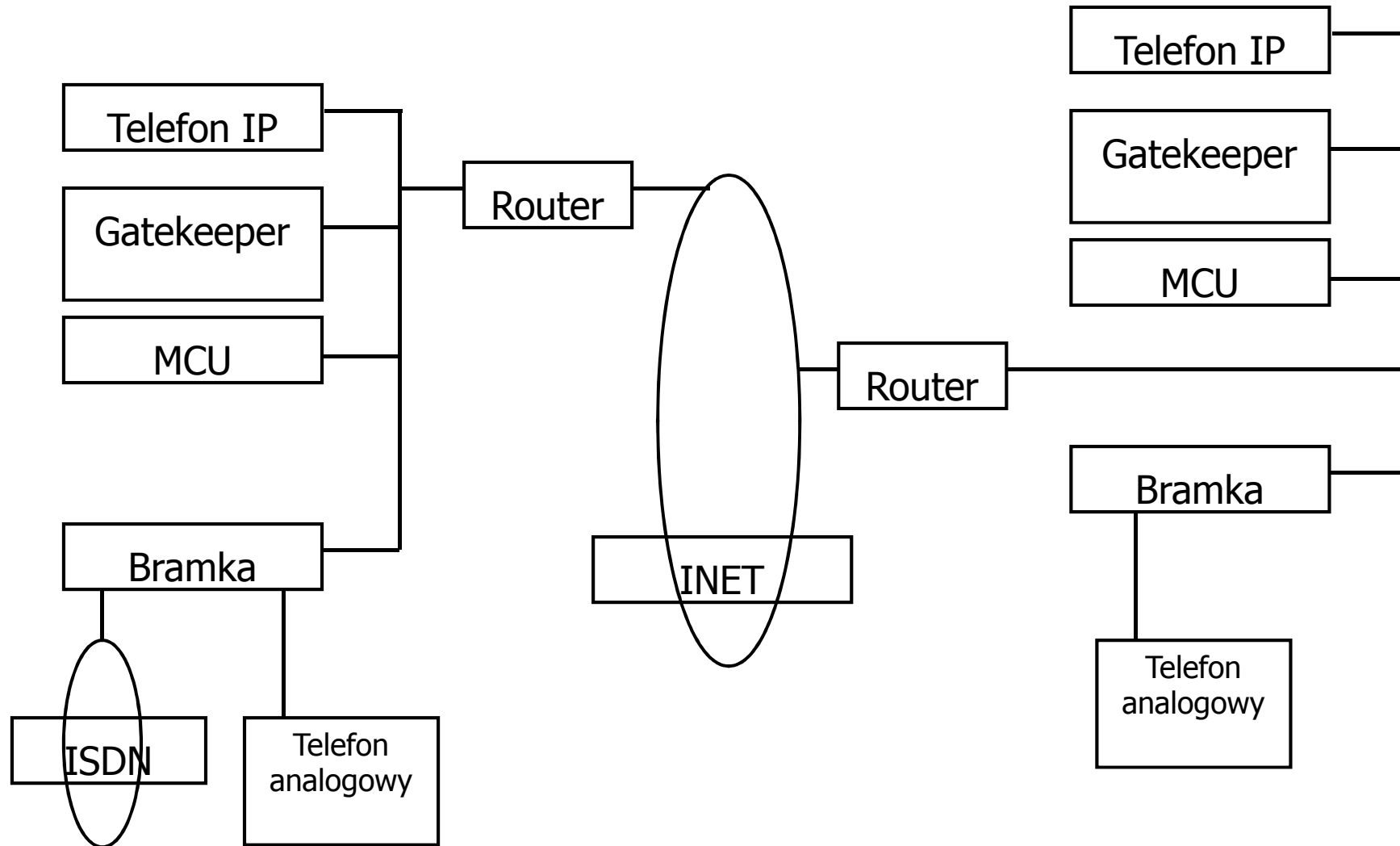
☒ Audio:

- ☒ G.711, G.729 , G.723.1 , G.722 , G.728

☒ Wideo:

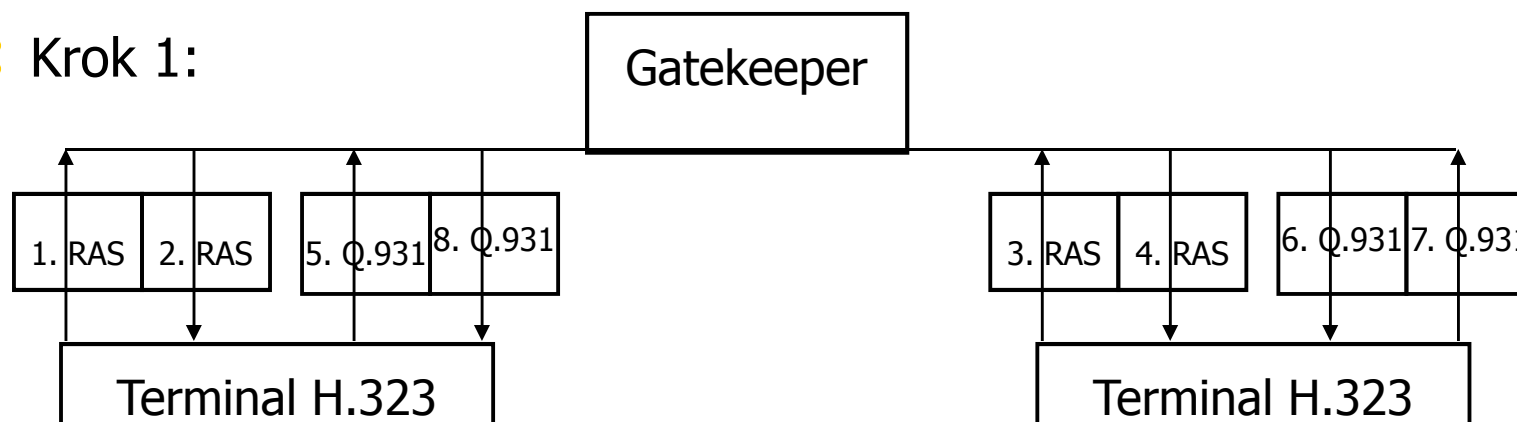
- ☒ H.261, H.263, H.264

H.323 - przykładowa architektura systemu

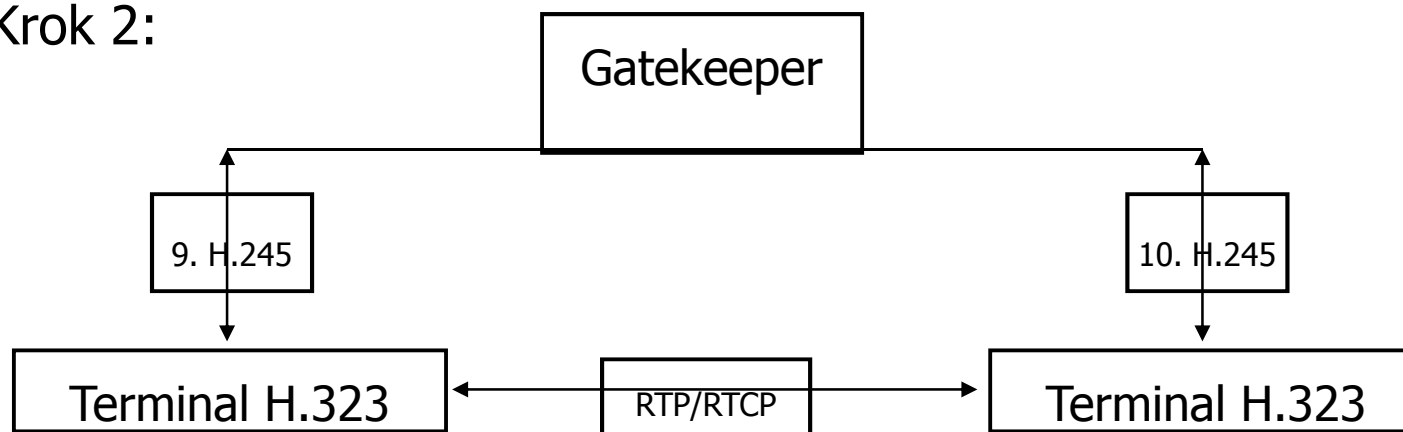


H.323 - procedura łączenia

⌘ Krok 1:



⌘ Krok 2:



Standard SIP



- ⌘ SIP (*Session Initiation Protocol*) – protokół służący do kontroli sesji komunikacyjnych opartych o multimedia (dźwięk wizja)
- ⌘ Komunikacja w ramach protokołu:
 - ⏏ Oparta o port 5060 dla SIP
 - ⏏ Oparta o port 5061 dla SIPS, czyli zabezpieczonej wersji szyfrowanej protokołu (szyfrowanie TLS)
- ⌘ SIP często działa we współpracy z protokołami RTP/Secure RTP do wspomagania transmisji strumieni multimedialnych, oraz SDP (*Session Description Protocol*).
- ⌘ Cele SDP: identyfikowanie stron komunikacji, zestawienie sesji, negocjowanie parametrów transmisji (numery portów, dalsze protokoły, kodeki). Cechy te zwane są *profilem* sesji.

Adresy SIP

- ⌘ Każdy element sieci SIP, taki jak user agent czy skrzynka wiadomości jest identyfikowana przy użyciu URI (*Uniform Resource Identifier*)
- ⌘ Typowa postać:
 - ☒ sip:uzytkownik:haslo@host:port
 - ☒ sips:uzytkownik:haslo@host:port
- ⌘ W przypadku SIP możliwe jest wprowadzenia adresacji opartej na numerach telefonów tłumaczonych przez infrastrukturę na powyższe adresy.
- ⌘ Endpoints nie muszą także korzystać z infrastruktury SIP lecz komunikują się (protokół SIP określany jest mianem peer-to-peer)

SIP – elementy sieci

⌘ User Agent (UA):

- ☒ Jest stroną komunikacji multimedialnej, przy czym może być:
 - ☒ *User Agent Client* (UAC) – wysyłającym żądania
 - ☒ *User Agent Server* (UAS) – odbierającym je
- ☒ Agentem może być przykładowo telefon SIP (SIP Phone) – sprzętowy lub programowy
- ☒ W SIP User Agent może się identyfikować przy pomocy tagu 'User-Agent' protokołu SIP (podobnie jak HTTP)

⌘ Proxy server:

- ☒ Pełni funkcje jednocześnie UAC oraz UAS
- ☒ Pośredniczy w komunikacji narzucając reguły filtrowania
- ☒ Wymaga rejestrowania od swoich klientów
- ☒ Przekazując żądania SIP dalej może wpływać na ich treść

SIP – elementy sieci (II)



- ⌘ Registrar – łączy adresy IP (jeden lub więcej) z URI. W nim rejestrują się UAC. Może zawierać rozbudowaną funkcjonalność dot. Identyfikacji klientów czy składowania danych opisujących klientów
- ⌘ Redirect server – konwertuje URI na inne URI (przekierowania)
- ⌘ Session border controller – kontroler wspomagający udostępnianie zasobów SIP do User Agents
- ⌘ Gateway – konwertuje komunikację z innej technologii (np. POTS – telefony analogowe)

Inne protokoły Internetu



⌘ Rozwiązania umożliwiające:

- ☒ Automatyczną konfigurację i wzajemną identyfikację hostów (alternatywnie do DHCP i Reverse ARP)
- ☒ Mapowanie i zdalne korzystanie z zasobów plikowych systemów operacyjnych
- ☒ Tunelowanie połączeń TCP, UDP
- ☒ Alternatywne do TCP i UDP usługi transportowe
- ☒ Zdalne wywoływanie procedur systemowych

SOCKS



- ⌘ Prosty protokół tunelowania ruchu nad IP (TCP lub UDP) przez serwer proxy (który pośredniczy).
- ⌘ Wersje protokołu:
 - ☒ SOCKS4 – podstawowa obecnie użytkowana
 - ☒ SOCKS4a – rozbudowana o resolver nazwy DNS hosta docelowego
 - ☒ SOCKS5 – wspiera UDP, IPv6
- ⌘ Interakcja na bazie protokołu SOCKS:
 - ☒ Klient wysyła żądanie połączenia dalej z podanie docelowego IP i portu, serwer odpowiada statusem wcześniej nawiązując połączenie z żądanym hostem docelowym. Od tego momentu serwer pośredniczy używając tego samego strumienia TCP lub asocjacji UDP.
 - ☒ Dwie opcje usługi: tylko połączenie (*connect*) lub gotowe do transmisji połączenie z otwarciem strumieni (*bind*)

NetBIOS (I)



- ⌘ NetBIOS (*Network Basic Input/Output System*)
- ⌘ Warstwa oprogramowania pierwotnie stworzona przez IBM i Sytek do połączenia sieciowego systemu operacyjnego ze sprzętem
- ⌘ Niski poziom realizacji sprzętowej - (przerwania CPU)
- ⌘ Zastosowanie w małych sieciach lokalnych
- ⌘ Adresowanie poprzez nazwy (16 znaków alfanumerycznych), poszukiwane techniką rozgłaszania
 - Indywidualne - unikatowe w ramach całej sieci. Przed wprowadzeniem nowej nazwy indywidualnej w sieci rozgłaszane jest kilkakrotnie zapytanie, czy taka nazwa gdzieś nie występuje.
 - Grupowe - możliwe jest wprowadzenie dodatkowej nazwy grupowej nawet jeśli istnieje już w sieci taka sama nazwa grupowa (kumulacja w grupie).

NetBIOS



⌘ Tryby komunikacji NetBIOS:

⏏ Dwa rodzaje trybów transmisji danych:

- ⏏ Bezpołączeniowy - szybszy, nie gwarantuje pomyślnej transmisji danych
- ⏏ Połączeniowy - ustanawiane jest połączenie (sesja) i zapewnia poprawną transmisję wiadomości o długości do 64kB

⏏ Wariant synchroniczny i asynchroniczny

- ⏏ Synchroniczny - wykonanie przerwania CPU i zatrzymanie przetwarzania do uzyskania odpowiedzi
- ⏏ Asynchroniczny - tylko wysłanie zlecenia do NetBIOS'u, wyniki odbierane później z inicjatywy aplikacji zlecającej

NetBEUI (I)



- ⌘ NetBEUI - *NetBIOS Extended User Interface*
- ⌘ Protokół transportowy (oparty na 3 i 4 warstwie OSI), niszowa alternatywa dla TCP/IP
- ⌘ Stworzony przez IBM, pojawił się na rynku komputerowym w 1985 roku
- ⌘ Przeznaczenie - małe sieci lokalne
- ⌘ Zapewnia usługi transportowe żądane przez NetBIOS
- ⌘ Został zaprojektowany do użytku tylko w sieciach LAN
- ⌘ Jego zadaniem jest zapewnienie niezawodności dostarczania pakietów w odpowiedniej kolejności pakietów

NetBEUI (II)



- ⌘ Rozgłoszeniowy charakter komunikacji - komunikaty trafiają do adresata poprzez rozgłoszenie.
- ⌘ Mimo wielu prób modyfikacji - nierutowalny. Pakiety trasowane do sieci zewnętrznych muszą być opakowane z użyciem innych protokołów (TCP/IP, IPX)
- ⌘ Tworzy wirtualne sesje (połączenie point-to-point). Maksymalna liczba sesji - 255
- ⌘ Współpracuje z wieloma systemami, jest między innymi integralną częścią systemów operacyjnych Windows for Workgroups, Windows NT i ich następców.

NFS (I)

- ⌘ NFS (*Network File System*), wprowadzony w roku 1984 przez firmę Sun Microsystems
- ⌘ Architektura klient-serwer, serwer NFS jest bezstanowy
- ⌘ Udostępnienie użytkownikom plików znajdujących się na innych komputerach bez konieczności ich kopiowania
- ⌘ Interfejs VFS (*Virtual File System*) zaimplementowany w jądrze systemu - klienta mapuje standardowe komendy lokalnego systemu na komendy dla serwera NFS na zdalnej maszynie
- ⌘ Funkcjonalność protokołu:
 - ⏏ Wyszukanie pliku w obrębie katalogu;
 - ⏏ Czytanie zbioru pozycji katalogowych;
 - ⏏ Manipulowanie katalogami;
 - ⏏ Dostęp do atrybutu pliku;
 - ⏏ Czytanie i pisanie plików

NFS (II)

- ⌘ V-węzły (*v-nodes*) systemu NFS - analogiczne do i-węzłów (*i-nodes*) systemu plików UNIX
- ⌘ Procedura wykonywania operacji:
 - ⏏ Klient inicjuje operację za pomocą zwykłej funkcji systemowej,
 - ⏏ Warstwa systemu operacyjnego odwzorowuje wywołanie tej funkcji na operację systemu VFS na odpowiednim v-węźle,
 - ⏏ Warstwa systemu VFS rozpoznaje plik jako zdalny i wywołuje odpowiednią procedurę NFS,
 - ⏏ Następuje wywołanie procedury zdalnej odnoszące się do warstwy obsługi systemu NFS dla zdalnego serwera,
 - ⏏ Wywołanie to przenika do warstwy VFS w systemie zdalnym, który identyfikuje je jako lokalne i wywołuje odpowiednią operację systemu plików,
 - ⏏ Ta sama droga jest odtwarzana przy przekazywaniu wyniku.

WINS



- ⌘ WINS - *Windows Internet Naming Service*
- ⌘ Serwis komunikacyjny bazujący na nazwach komputerów
- ⌘ Niezależność od podsieci IP
- ⌘ Serwis oparty jest na serii komunikatów rozgłoszeniowych - rozgłoszenie adresu IP oraz nazwy
- ⌘ Informacje komputerach są dynamicznie aktualizowane i nie wymagają interwencji człowieka (przeciwnie do DNS). Istnieje jednak opcja statycznych WINS (konfigurowane ręcznie)
- ⌘ Klient musi znać adres IP serwera WINS oraz najlepiej adres serwera zapasowego
- ⌘ Realizacja: usługa Win NT, konfigurowana poprzez narzędzie *WINS Manager*

SMB



- ⌘ SMB (*Server Message Block*) – protokół wykorzystywane do zdalnego udostępniania zasobów systemowych (pliki, urządzenia). Inna nazwa: CIFS (*Common Internet File System*)
- ⌘ Protokół nadbudowany nad NetBIOS i TCP lub NetBEUI
- ⌘ Popularne narzędzie (Serwer Plików) korzystające z SMB: SAMBA. Ponadto SMB jest szeroko wykorzystywany przez systemy operacyjne firmy Microsoft
- ⌘ Funkcjonalność:
 - ☒ Współdzielenie katalogów
 - ☒ Otwarcie – zamknięcie pliku
 - ☒ Otwarcie – zamknięcie pliku drukarskiego
 - ☒ Czytanie – zapis pliku
 - ☒ Połączenie i rozłączenie od pliku lub usługi drukarskiej
 - ☒ Tworzenie – usuwanie pliku, katalogu
 - ☒ Pobieranie i ustawianie atrybutów
 - ☒ Blokowanie – odblokowywanie zakresu bitowego w pliku

Remote Procedure Call (I)



- ⌘ Wysyłanie datagramu UDP i oczekiwanie na odpowiedź, która miała by nadejść po zdalnym wykonaniu czynności zleconych za jego pośrednictwem (gdy tak założono w logice funkcjonowania aplikacji) może być ekwiwalentem tzw. zdalnego wywołania procedury (RPC – *Remote Procedure Call*). Istnieje nomenklatura opisująca elementy takiego procesu (przydatna przy implementacjach)
- ⌘ Z punktu widzenia aplikacji (zarówno klienta jak i serwera) komunikacja związana z RPC przebiega lokalnie – aplikacja (*klient*) wywołuje procedurę biblioteczną dostarczając parametry lub przekazuje modułowi bibliotecznemu referencję do własnej procedury, która mogłaby być wywołana (*server*).
- ⌘ Hosty implementujące RPC udostępniają zmodyfikowane wersje bibliotek – powodujące przekazanie wywołania/odpowiedzi zdalnie przez sieć

Remote Procedure Call (II)

- ⌘ Aplikacja klienta wywołuje lokalnie procedurę biblioteczną określaną jako *client stub*. Procedura ta nie jest jednak realizowana lokalnie – w zamian za to przez sieć zostaje wysłane zlecenie wykonania zdalnego i trafia ono do przeciwnego hosta. Tam jest przekazywane do biblioteki utrzymującej tzw. *server stub* – procedurę faktycznie zaimplementowaną i zarejestrowaną przez aplikację serwera. Status wykonania procedury trafia do aplikacji klienta analogiczną drogą.
- ⌘ Format komunikatów wykorzystywanych przez RPC najczęściej musi uwzględnić przekazanie parametrów wykonania procedury. Proces upakowania takich parametrów w komunikacie to *marshalling*.



Modele komunikacji sieciowej



- ⌘ Zdefiniowane aby porządkować architekturę systemów rozproszonych, a w szczególności techniki komunikowania się pomiędzy ich fizycznymi komponentami hostami i procesami
- ⌘ Najbardziej popularne modele:
 - ⌘ Peer to Peer
 - ⌘ Mailslots
 - ⌘ Named Pipes
 - ⌘ Client-Server
- ⌘ W ramach danego modelu dopuszcza się często powielanie bazowej konstrukcji z powyższej listy, tworząc w middleware sieci wielowęzłowe

Usługi sieciowe w modelu Peer to Peer



- ⌘ Model tzw. sieci partnerskich (*Peer to Peer Network*)
- ⌘ Sieć typu „każdy z każdym” - wszystkie uczestniczące w sieci komputery mają podobny stopień kontroli nad zasobami
- ⌘ Współdzielenie np. przestrzeni dyskowej i drukarek, czasu procesora
- ⌘ Zastosowanie dla małych grup roboczych (sieci odizolowane), przeznaczonych do realizacji spójnych zadań
- ⌘ Przykłady: Microsoft Windows for Workgroups, IBM LAN Server, Artisoft LANtastic, Novell Personal NetWare)

Usługi sieciowe w modelu Mailslots



- ⌘ MSFS (*Mailslot File System*),
- ⌘ Jednokierunkowy transfer danych (do adresata)
- ⌘ Adresacja hierarchiczna, dostęp przez nazwę:
 - ⏏ \\DomainName\mailslot\[path]name
- ⌘ Każdy mailslot posiada właściciela (proces odbierający)
- ⌘ Dwie klasy mailslotów:
 - ⏏ Pierwsza - zorientowana połączeniowo, transfer danych dla w trybie jeden-do-jeden, lub jeden-do-wielu,
 - ⏏ Druga - oparta na datagramach, transfer nie gwarantowany, tryb jeden-do-jeden, lub jeden-do-wielu

Usługi sieciowe z użyciem Named Pipes

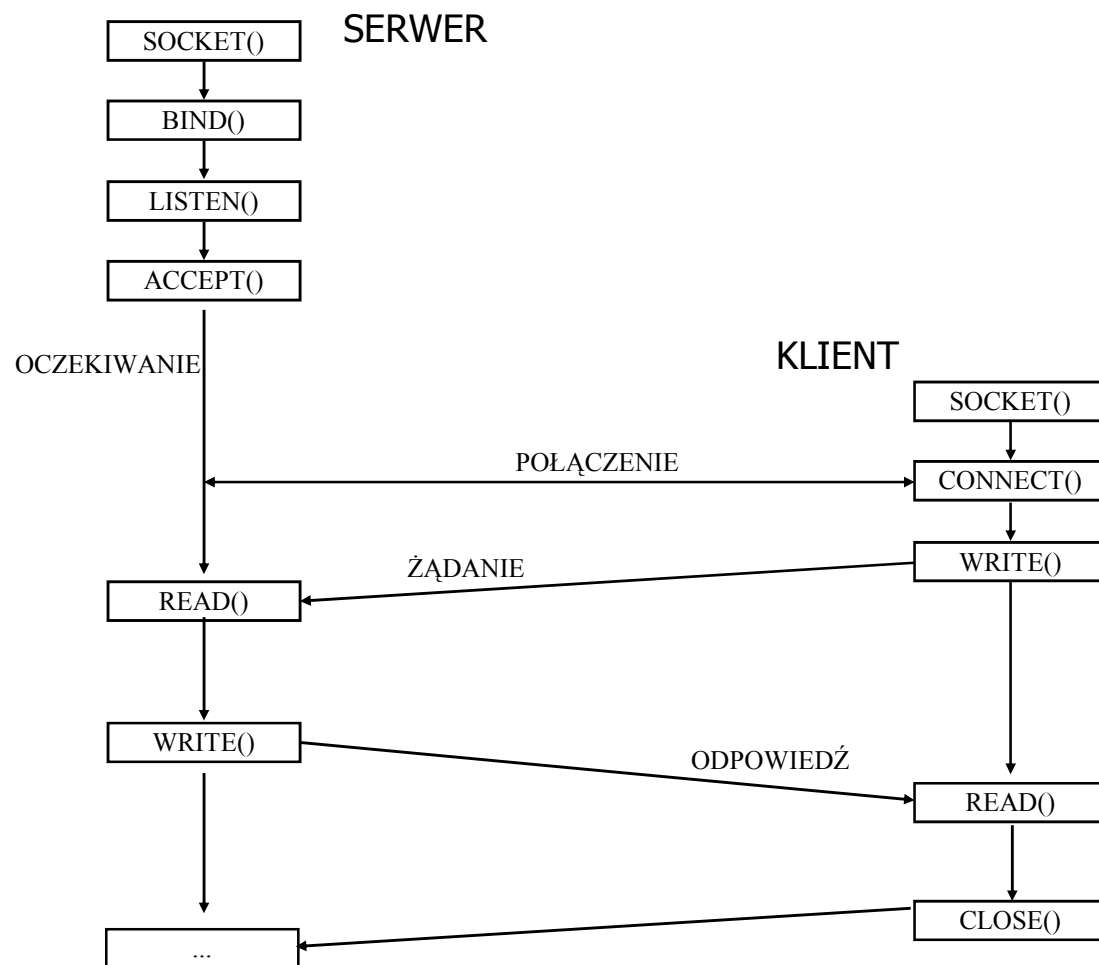
- ⌘ NPFS (Named Pipe File System),
- ⌘ Pipe jest sekcją dzielonej pamięci, używanej przez proces do komunikacji,
- ⌘ Adresacja hierarchiczna, dostęp przez nazwę:
 - ☒ \\ServerName\pipe\PipeName
- ⌘ Dostępne połączenie: jeden-do-jeden
- ⌘ Komponenty:
 - ☒ Pipe server - proces, który tworzy potok
 - ☒ Pipe client - łączy się z gotowym potokiem
- ⌘ Serwer potoku nie może tworzyć tego potoku na innym komputerze, jednak klient może komunikować się zdalnie
- ⌘ Istnieją potoki anonimowe - nie są wtedy przewiązane z żadnym procesem tworzącym.

Usługi sieciowe w modelu Client-Server

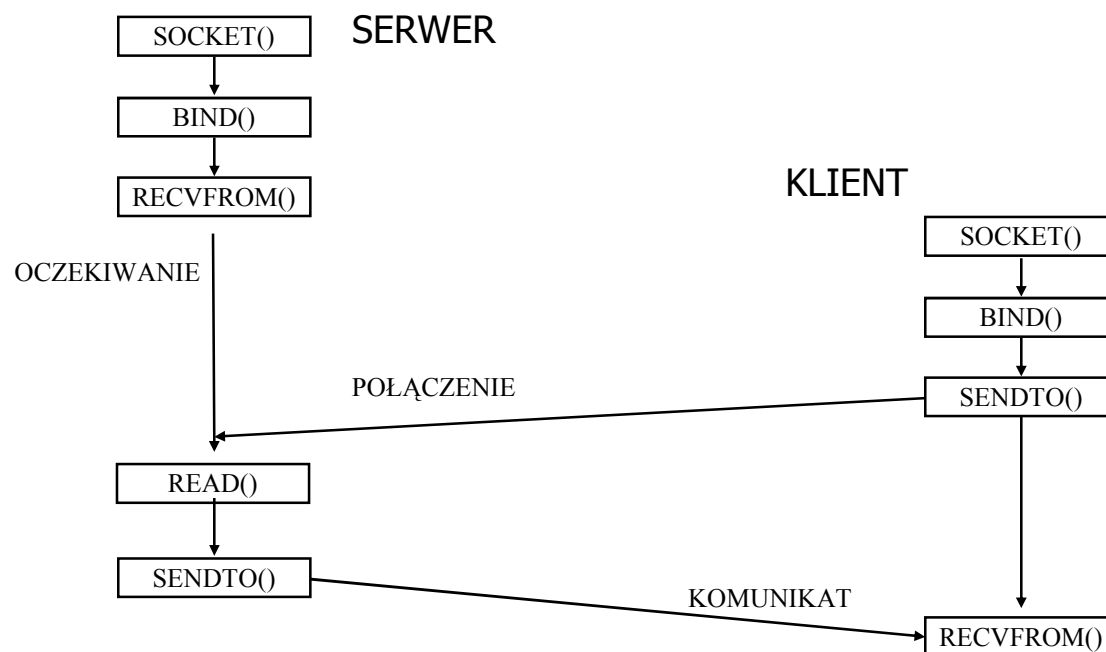


- ⌘ Model podstawowy: dwie stacje, jedna świadczy usługi, druga korzysta z usług
- ⌘ Zalety i wady:
 - ⏏ Łatwa realizacja i przejście z systemu rozproszonego
 - ⏏ Obciążenie zadaniami, pochodzące od oprogramowania aplikacyjnego, rozłożone jest na wiele maszyn
 - ⏏ Mały ruch w sieci, gdyż serwer przekazuje klientowi jedynie żadaną informację
 - ⏏ Dane zgromadzone w jednym miejscu - łatwość kontroli nad danymi systemu
 - ⏏ Łatwość stosowania mechanizmów monitorowania i kontroli dostępu
 - ⏏ Problemy z kolizjami przy edycji wspólnych danych

Client-Server - scenariusz trybu połączeniowego



Client-Server - scenariusz trybu bezpołączeniowego



Client-Server - przykład kodu serwera (Java)

```
⌘ ServerSocket socket;  
⌘ Socket klient;  
⌘ BufferedReader net_in;  
⌘ PrintWriter net_out;  
⌘ try {  
⌘     socket = new ServerSocket (2222);  
⌘     while (true) {  
⌘         klient = socket.accept ();  
⌘         net_in = new BufferedReader (new InputStreamReader(klient.getInputStream()));  
⌘         net_out = new PrintWriter (new OutputStreamWriter(klient.getOutputStream()));  
⌘         String s =net_in.readLine ();  
⌘         net_out.println ("Przyslales: " + s);  
⌘         net_out.flush ();  
⌘         net_in.close ();  
⌘         net_out.close ();  
⌘     }// while  
⌘ } catch (IOException e) {}
```

Client-Server - przykład kodu klienta (Java)

```
⌘ Socket klient;  
⌘ BufferedReader net_in;  
⌘ PrintWriter net_out;  
⌘ try {  
⌘     socket = new Socket („200.200.200.200”, 2222);  
⌘     net_in = new BufferedReader (new InputStreamReader(socket.getInputStream ()));  
⌘     net_out = new PrintWriter (new OutputStreamWriter(socket.getOutputStream()),true);  
⌘     net_out.println („Test...”);  
⌘     net_out.flush ();  
⌘     String s = net_in.readLine();  
⌘     System.out.println („Odebrana odpowiedz: „ + s);  
⌘     net_in.close ();  
⌘     net_out.close ();  
⌘     socket.close ();  
⌘ } catch (IOException e) {}
```